AtaEtzel — Claude Code İçin SITL Veri Üretimi Promptları

Nov 4, 2026 · @Holdbane

Bu belge ne işe yarar?

Bu belge, SITL veri üretim reposunu Claude Code'a yazdırman için hazırlanmış, sırayla kopyala-yapıştır yapacağın promptları içerir. Sen kod yazmıyorsun; her adımda hazır metni Claude Code'a veriyorsun, o kodu üretiyor, sen de o makinede test ediyorsun.

Nasıl kullanılır

1. Boş repoyu bir bilgisayara klonla ve içinde terminal aç: `git clone <repo-adresin> && cd <repo-klasörü>`.
2. O klasörde Claude Code'u başlat: `claude`.
3. **Önce "Adım 0" (proje bağlamı) bloğunu** birebir yapıştır. Bu, Claude Code'a projenin ne olduğunu ve kuralları öğretir; ayrıca repoya bir `CLAUDE.md` yazar. Bundan sonraki her adım bu bağlama dayanır.
4. Sonra **Adım 1'den 7'ye kadar** sırayla git. Her adımı yapıştır, Claude Code kodu üretsin, sonra o adımın altındaki **"Test"** kutusundaki komutu çalıştır.
5. Hata alırsan hatanın tam çıktısını Claude Code'a yapıştır ve "bu hatayı düzelt" de. Takılırsan çıktıyı bana da gönder.

Önemli sıra kuralı

Adımlar birbirine bağlı: proxy (Adım 2) olmadan uçuş kaydı alınamaz, kayıt olmadan etiketleme (Adım 5) çalışmaz. Sırayı bozma. Bir adım tam çalışmadan sonrakine geçme.

Kim ne yapsın?

- **Kurulumu ve ilk testi bir kişi (örneğin sen) tek makinede baştan sona yapsın.** Repo çalışır hale gelince diğer iki kişi yalnızca klonlayıp `setup.sh` çalıştırır.
- Böylece "bende çalışmıyor" sorunlarını tek makinede çözmüş olursun.

Bir uyarı

Claude Code bazen istenenden fazlasını yapar veya dosya adlarını değiştirir. Her adımda “sadece şunu yap, başka dosyaya dokunma” dersen daha düzenli ilerler. Dosya ve sütun adları bir sonraki bölümdeki sözleşmeye birebir uymalı; promptlarda bunu zaten belirttim.

Üç bilgisayar düzeni ve veri şeması

Bütün betikler aynı dosya adlarını ve aynı sütunları kullanmalı; yoksa parçalar birbirine bağlanmaz. Bu bölüm, promptların hepsinin dayandığı ortak sözleşmedir. Claude Code'a "Adım 0" ile bunu veriyoruz; burası senin ve ekibin başvuru kaynağı.

Pencere ve uçuş numarası (çakışmayı önler)

Her bilgisayar ayrı bir numara aralığı kullanır; böylece üç kişi aynı anda üretim yapınca dosya adları çakışmaz.

Bilgisayar	`pc_id`	Uçuş numarası aralığı	Örnek
1. PC	pc1	F100–F199	F101_normal
2. PC	pc2	F200–F299	F205_spoof_cmd
3. PC	pc3	F300–F399	F312_flood

`pc_id` ve bu aralık `config.yaml` dosyasında tutulur (Adım 1). Her kişi kendi makinesinde bu dosyada yalnızca `pc_id`'yi ayarlar.

Klasör yapısı

```
veri/
  ucuslar/
    F101_normal/
      paketler.parquet      (her iki yöndeki tüm paketler)
      ham/flight.tlog       (MAVProxy tlog, isteğe bağlı)
      ham/sitl.BIN          (SITL DataFlash günlüğü, fizik referansı)
      meta.json
    F205_spoof_cmd/
      paketler.parquet
      meta.json
      ground_truth.json     (yalnızca saldırılı uçuşlarda)
  bolme.csv                 (flight_id, split)
  oznitelikler/
    pencereler.parquet      (tüm uçuşların 1 sn'lik öznitelikleri)
```

paketler.parquet sütunları

Sütun	Tip	Açıklama
t_ns	int64	Alınma zamanı, nanosaniye (`time.time_ns()`)
direction	string	`gcs2uav` veya `uav2gcs`
src_ip	string	Gönderenin IP'si — yalnızca etiketleme için
src_port	int	Gönderenin portu — yalnızca etiketleme için
sysid	int	MAVLink sistem kimliği
compid	int	MAVLink bileşen kimliği
seq	int	Sıra numarası (0–255)
msgid	int	Mesaj türü numarası
msgname	string	Mesaj türü adı (HEARTBEAT, COMMAND_LONG…)
length	int	Paket uzunluğu (bayt)
is_signed	bool	İmzalı mı
crc_ok	bool	CRC doğru mu

Sütun	Tip	Açıklama
fields_json	string	Çözümlenmiş alanlar (JSON)
raw_hex	string	Ham bayt (hex) — yalnızca tekrar oynatma ve etiketleme için

Kritik kural: `src_ip`, `src_port` ve `raw_hex` **asla öznitelik olmaz**. Yalnızca etiketlemede ve replay saldırısında kullanılır. Modele verilirse veri sızıntısı olur (model saldırıyı değil, saldırganın portunu öğrenir).

meta.json alanları

`flight_id`, `pc_id`, `ardupilot_version`, `vehicle` (ArduPlane), `mission_file`, `wind` (hız/yön), `location`, `date`, `operator`, `duration_s`, `notes`.

ground_truth.json alanları (yalnızca saldırılı uçuşlar)

`flight_id`, `attack_type` (spoof_cmd / impersonation / replay / flood / fake_telemetry), `start_ns`, `end_ns`, `attacker_addr`, `params` (saldırıya özgü ayarlar).

pencereler.parquet (1 sn'lik öznitelikler)

`flight_id`, `window_start_ns`, sonra öznitelik sütunları: toplam paket sayısı, mesaj türüne göre sayılar, farklı sysid sayısı, seq atlama ve tekrar sayısı, paketler arası sürenin ortalaması ve std'si, komut mesajı sayısı; en sonda `label` (normal veya saldırı türü) ve `attack_ratio` (penceredeki saldırı paketi oranı).

Adım 0 — Proje bağlamı (ilk mesaj)

Claude Code'a **ilk** bunu yapıştır. Projeyi tanıtır ve repoya bir `CLAUDE.md` yazar; sonraki adımlar bu bağlamı kullanır.

```
Merhaba. Bu repoda bir bitirme projesinin SITL veri üretim hattını kuracağız.
Önce sana projeyi anlatıyorum, sonra adım adım kod isteyeceğim. Şimdilik KOD
YAZMA; yalnızca projeyi öğren ve bir CLAUDE.md oluştur.

PROJE: "Sabit Kanatlı İHA'lar için MAVLink Haberleşmesine Yönelik Siber
Saldırı Tespit ve Müdahale Sistemi" (ekip: AtaEtzel). Uzun vadeli hedef,
telsiz ile otopilot arasına giren bir MAVLink güvenlik duvarı. BU REPO ise
yalnızca VERİ ÜRETİMİ kısmıdır: ArduPlane SITL simülasyonunda normal ve
```

```
saldırılı uçuşlar üretip, MAVLink trafiğini kaydedip, etiketleyip, öznitelik
çıkarmak.

ORTAM: Ubuntu 22.04 ve 24.04. Kod üç farklı bilgisayarda BİREBİR AYNI
çalışmalı. ArduPlane sürümü Plane-4.7.2'ye sabitlenecek. Tüm belgeler, kod
yorumları, commit mesajları ve kullanıcıya giden mesajlar TÜRKÇE olacak.

MİMARİ (simülasyonda):
  Yer istasyonu (MAVProxy) ve Saldırgan betikleri  --UDP 14550-->  [KAYIT
PROXY'Sİ]  --TCP 5760-->  ArduPlane SITL
  SITL'den gelen telemetri de proxy üzerinden kanaldaki herkese geri gider.
  UDP 14550 ortak telsiz kanalını, TCP 5760 otopilota giden kabloyu temsil
eder. Proxy iki yöndeki her paketi zaman damgasıyla kaydeder. Bu proxy
ileride güvenlik duvarının iskeleti olacak.

BEŞ SALDIRI TÜRÜ: sahte komut (spoof_cmd), kimlik taklidi (impersonation),
tekrar oynatma (replay), boğma (flood), sahte telemetri (fake_telemetry).

KESİN KURALLAR:
1. Saldırı betikleri YALNIZCA 127.0.0.1 (localhost) hedefine bağlanabilir;
kodda bu kontrol zorunlu, aksi halde betik hata verip çıksın. Bunlar yalnızca
kendi simülasyonumuz içindir.
2. Ham uçuş verisi (parquet, tlog, BIN) git'e KONMAZ; .gitignore ile
dışlanır. Repoda yalnızca kod, belge ve çok küçük örnek veri olur.
3. Dosya ve sütun adları, sana birazdan vereceğim "VERİ ŞEMASI"na birebir
uyacak. Kendiliğinden ad değiştirme.
4. Her betik `--help` ile açıklama versin ve başında Türkçe bir yorum bloğu
bulunsun.
5. Yalnızca sana söylediğim dosyaları oluştur/düzenle; başka dosyalara
dokunma.

[BURAYA, bu belgedeki "Üç bilgisayar düzeni ve veri şeması" bölümünün
tamamını kopyala-yapıştır.]

Şimdi: (1) yukarıdaki her şeyi özetleyen bir CLAUDE.md oluştur, (2) bana
projeyi doğru anladığını 3-4 cümleyle teyit et. Kod yazma.
```

Senin yapacağın: Köşeli parantezli yere, bu belgedeki “Üç bilgisayar düzeni ve veri şeması” bölümünü olduğu gibi yapıştır. Claude Code "anladım" dedikten sonra Adım 1'e geç.

> **Test:** Repoda bir `CLAUDE.md` oluştu mu, içinde Plane-4.7.2 ve localhost kuralı geçiyor mu? `cat CLAUDE.md` ile bak.

Adım 1 — Repo iskeleti, kurulum betiği ve config

```
Artık kod yazabilirsin. Şu dosyaları oluştur, başka bir şeye dokunma:

1. Klasör iskeleti: betikler/ (python kodları), gorevler/ (waypoint
dosyaları), belgeler/ (rehberler), veri/ (ham veri, git'e girmeyecek). veri/
içine .gitkeep koy.

2. .gitignore: veri/ucuslar/, veri/oznitelikler/, *.parquet, *.tlog, *.BIN,
__pycache__/, *.pyc, venv/, .venv/ dışlansın. Ama veri/ornek/ altındaki küçük
örnek dosyalar git'e GİRSİN (bunu .gitignore'da ! ile istisna yap).

3. config.yaml: pc_id (varsayılan pc1), ucus_no_araligi (pc1 için 100-199),
host (127.0.0.1), udp_port (14550), tcp_port (5760), sitl_port_base (5760),
veri_klasoru (veri/ucuslar). Her alanın yanına Türkçe açıklama yorumu.

4. requirements.txt: pymavlink, mavproxy, pandas, numpy, pyarrow, pyyaml.
Sürümleri sabitleme, en güncel kararlıları bırak.

5. betikler/ortak.py: tüm betiklerin kullanacağı yardımcılar — config.yaml
okuma, uçuş klasörü yolu üretme, zaman damgası, basit Türkçe loglama. Bir
sonraki uçuş numarasını config'teki aralığa göre otomatik bulan bir fonksiyon
da olsun (o aralıkta en büyük mevcut numaradan bir fazlası).

6. setup.sh: Ubuntu 22.04 ve 24.04'te çalışan, baştan sona kurulum betiği:
   - apt bağımlılıkları (git, python3, python3-venv, build-essential vb.)
   - ArduPilot'u Plane-4.7.2 etiketinde ~/ardupilot'a klonlama (zaten varsa
atlama), submodule'leri çekme
   - ArduPilot'un kendi Tools/environment_install/install-prereqs-ubuntu.sh -
y betiğini çalıştırma (MAVProxy ve pymavlink'i bu kurar)
   - ./waf configure --board sitl && ./waf plane ile derleme
   - Bu repo için bir python sanal ortamı (.venv) oluşturup
requirements.txt'i kurma
   - Her adımda Türkçe ilerleme mesajı; bir adım başarısız olursa açık Türkçe
hata verip durma (set -e)
   - Sonunda "kurulum tamam" ve sonraki adımı söyleyen bir özet

7. betikler/kontrol.py: kurulumdan sonra çalıştırılacak bir sağlık kontrolü —
ArduPlane derlenmiş mi, pymavlink ve mavproxy import ediliyor mu, config.yaml
okunuyor mu, portlar boş mu; hepsini Türkçe "TAMAM/HATA" olarak yazsın.

Kodları PEP8'e uygun, bol Türkçe yorumlu yaz.
```

> **Test:** `bash setup.sh` çalıştır (15–30 dk sürer, ArduPilot derlenir). Bitince `source .venv/bin/activate && python betikler/kontrol.py` — hepsi TAMAM olmalı. Hata çıkarsa çıktıyı Claude Code'a yapıştır.

Adım 2 — Kayıt proxy'si

Bu projenin kalbi. Yavaş ilerle; bu adımı iyi test et.

```
betikler/kayit_proxy.py oluştur. Görevi: yer istasyonu/saldırgan ile
ArduPlane SITL arasına girip iki yöndeki her MAVLink paketini kaydetmek.

DAVRANIŞ:
- config.yaml'dan host ve portları oku. Bir UDP soketi 14550'de dinler (yer
istasyonu ve saldırgan buraya yazar). SITL'ye TCP 5760 ile bağlanır.
- gcs2uav yönü: UDP 14550'den gelen paketi SITL'ye (TCP 5760) ilet.
- uav2gcs yönü: SITL'den gelen paketi, 14550'ye yazmış olan tüm adreslere
geri gönder (böylece saldırgan da telemetriyi dinleyebilir, gerçek hayattaki
gibi).
- Komut satırı argümanı: --ucus F101_normal (uçuş adı). Argman verilmezse
config'teki aralıktan sonraki normal numarayı üret.
- Her paketi, VERİ ŞEMASI'ndaki paketler.parquet sütunlarıyla kaydet.
pymavlink ile paketi çözümle (mavutil / mavlink2); sysid, compid, seq, msgid,
msgname, length, is_signed, crc_ok, fields_json, raw_hex doldur.
src_ip/src_port geldiği UDP adresinden al. SITL'den gelen yönde
src_ip/src_port'u otopilotun adresi yaz.
- Performans: paketleri önce bir tampona/listeye al, periyodik olarak (ör.
her 2 sn veya her 500 paket) parquet'e ekle; her pakette diske yazma. Program
temiz kapanırken (Ctrl+C) kalan tamponu da yaz.
- Uçuş klasörünü (veri/ucuslar/<ucus>/) oluştur; paketler.parquet oraya yaz.
meta.json'u da başlat (pc_id, ardupilot_version, tarih, ucus_id; süre ve
diğerleri kapanışta güncellensin).
- Bol Türkçe log: "14550 dinleniyor", "SITL'ye bağlandı", saniyede yaklaşık
kaç paket aktığı vb.
- raw_hex'i parquet'te tutmak dosyayı büyütür; config.yaml'a raw_hex_kaydet
(varsayılan true) diye bir anahtar ekle; replay saldırısı ve etiketleme buna
ihtiyaç duyar.

Sadece bu dosyayı ve gerekiyorsa ortak.py'ye küçük bir yardımcı ekle. Başka
dosyaya dokunma.
```

> **Test (üç terminal):**
>
> 1. `cd ~/ardupilot/ArduPlane && sim_vehicle.py -v ArduPlane --no-mavproxy` (SITL, 5760'ta bekler)
> 2. `python betikler/kayit_proxy.py --ucus test01` (proxy)
> 3. `mavproxy.py --master=udpout:127.0.0.1:14550 --console` (yer istasyonu)
>
> MAVProxy konsolunda `mode`, `arm throttle`, `mode TAKEOFF` gibi komutlar ver. Birkaç dakika sonra proxy'yi Ctrl+C ile kapat.
> `veri/ucuslar/test01/paketler.parquet` oluştu mu? Python'da oku: `import pandas; df=pandas.read_parquet('veri/ucuslar/test01/paketler.parquet'); print(df.shape); print(df.msgname.value_counts().head()); print(df.direction.value_counts())` — iki yön de ve HEARTBEAT görünmeli.

Adım 3 — Uçuş otomasyonu ve görevler

```
İki şey oluştur: görev dosyaları ve otomatik uçuş betiği.

A) gorevler/ klasörüne 5 waypoint dosyası (.waypoints, QGC WPL 110 biçimi),
sabit kanatlı ArduPlane için, ilk öğe NAV_TAKEOFF (komut 22) olacak:
   - gorev1_dikdortgen.waypoints: basit dikdörtgen rota, ~100 m irtifa
   - gorev2_uzunhat.waypoints: uzun düz hat ve geniş dönüş
   - gorev3_loiter.waypoints: bir noktada LOITER ağırlıklı, farklı irtifa
(~150 m)
   - gorev4_karisik.waypoints: birkaç ara nokta + mod değişimi için uygun
   - gorev5_farkli.waypoints: farklı rota ve yön
   Varsayılan SITL konumu (CMAC) koordinatlarını kullan.

B) betikler/ucus_yap.py: bir normal uçuşı baştan sona otomatik yapan betik.
pymavlink ile proxy'ye (udpout:127.0.0.1:14550) bağlanır — yani YER İSTASYONU
gibi davranır, doğrudan SITL'ye değil, proxy üzerinden. Adımlar:
   - Argümanlar: --gorev gorevler/gorev1_dikdortgen.waypoints, --ucus (isteğe
bağlı, yoksa otomatik numara), --ruzgar-hiz, --ruzgar-yon (isteğe bağlı), --
sure (saniye).
   - Görevi yükle (MISSION_COUNT/MISSION_ITEM), rüzgârı ayarla (param set
SIM_WIND_SPD / SIM_WIND_DIR), arm et, AUTO moduna al, kalkışı başlat.
   - Belirtilen süre boyunca veya görev bitene kadar uçur; bu sırada
meta.json'a rüzgâr, görev, konum bilgisini yaz.
   - --speedup KULLANMA (zamanlamayı bozar, 1 kalsın); bunu kod yorumunda
açıkla.
   - Sonunda Türkçe özet: hangi görev, kaç saniye, hangi rüzgâr.
   NOT: Bu betik proxy'nin ZATEN çalışıyor olmasını bekler; başında bunu
kontrol et ve değilse Türkçe uyar.

C) betikler/normal_uretim.sh: proxy'yi ve ucus_yap.py'yi birlikte çalıştırıp
tek komutla bir normal uçuş üreten yardımcı betik (SITL'nin ayrı terminalde
çalıştığını varsay). Hangi sırayla ne çalıştığını Türkçe yazsın.
```

> **Test:** SITL'yi başlat. `python betikler/kayit_proxy.py --ucus F101_normal` bir terminalde, `python betikler/ucus_yap.py --gorev gorevler/gorev1_dikdortgen.waypoints --ucus F101_normal --sure 180` başka terminalde. Uçak kalkıp rotayı uçmalı; sonunda `veri/ucuslar/F101_normal/` içinde dolu bir parquet ve meta.json olmalı.

Adım 4 — Saldırı betikleri ve ground truth

```
betikler/saldirilar/ klasörüne 5 saldırı betiği oluştur. HEPSİ ŞU KURALLARA
UYACAK:
- Yalnızca 127.0.0.1'e bağlanır; hedef farklıysa Türkçe hata verip çıkar
(guvenlik kontrolü, ortak.py'de tek fonksiyon).
- Proxy'nin UDP 14550 portuna yazar (yani telsiz kanalına girer), doğrudan
SITL'ye değil.
- Çalışırken ilgili uçuş klasörüne ground_truth.json yazar/ekler:
attack_type, start_ns, end_ns, attacker_addr, params. Birden fazla saldırı
aynı uçuşta olabilir; liste olarak tut.
- Ortak argümanlar: --ucus (hangi uçuş), --sure (saldırı kac saniye), --
baslangic-gecikme (uçuş başladıktan kaç sn sonra).
- Bol Türkçe log ve başında ne yaptığını anlatan yorum bloğu.

BETİKLER:
1. spoof_cmd.py — sahte komut: COMMAND_LONG ile mod değişikliği veya disarm
gibi tehlikeli bir komut gönderir (sysid'i meşru yer istasyonu gibi, ör.
255).
2. impersonation.py — kimlik taklidi: kendini yer istasyonu (sysid 255) gibi
gösterip sahte HEARTBEAT ve komut akışı üretir; meşru akışla çakışan
seq'lerle.
3. replay.py — tekrar oynatma: daha önce kaydedilmiş bir uçuşın
paketler.parquet'indeki raw_hex'ten gerçek komut paketlerini okuyup aynen
yeniden gönderir (--kaynak-ucus argümanı).
4. flood.py — boğma: yüksek frekanslı (ör. saniyede yüzlerce) HEARTBEAT veya
PING mesajı gönderip kanalı doldurur; --hiz (paket/sn) argümanı.
5. fake_telemetry.py — sahte telemetri: yer istasyonuna sahte
GLOBAL_POSITION_INT gönderir (gerçek konumdan sapmış); operatörü yanıltır.

Ayrıca betikler/saldiri_uret.sh: bir normal uçuşı tekrar uçururken ortasında
seçilen saldırıyı çalıştıran, tek komutla saldırılı uçuş üreten yardımcı
betik (--saldiri spoof_cmd --gorev ... gibi). Saldırıdan önce ve sonra temiz
bölüm kalsın.
```

> **Test:** Bir saldırılı uçuş üret: SITL + proxy (`--ucus F106_spoof_cmd`) + `ucus_yap.py` çalışırken, 60. saniyede `python betikler/saldirilar/spoof_cmd.py --ucus F106_spoof_cmd --sure 30` . Sonunda uçuş klasöründe `ground_truth.json` oluşmalı, içinde start_ns/end_ns dolu olmalı. Proxy'nin parquet'inde o aralıkta saldırgan adresinden paketler görünmeli.
>
> **Güvenlik testi:** Betiklerden birini sahte bir IP ile çalıştırmayı dene (ör. koda geçici 192.168.1.5 yaz); Türkçe hata verip çıkmalı. Bu, betiklerin dış ağa çıkamayacağını kanıtlar.

Adım 5 — Etiketleme, bölme, öznitelik, doğrulama

```
betikler/ altına dört işleme betiği oluştur. Hepsi VERİ ŞEMASI'na birebir
uysun.

1. etiketle.py: bir uçuşun paketler.parquet'ini ve ground_truth.json'unu
okur. Her pakete etiket ekler:
   - Paketin src_ip/src_port'u ground_truth'taki attacker_addr ile eşleşiyor
VE t_ns, [start_ns, end_ns] aralığındaysa label = attack_type; değilse label
= normal.
   - ground_truth.json yoksa tüm paketler normal.
   - Etiketli paketler.parquet'i aynı yere yazar (label sütunu ekler).
   - UYARI olarak koda yaz: src_ip/src_port yalnızca burada kullanılır,
öznitelik değildir.

2. oznitelik.py: bir veya tüm uçuşların etiketli paketlerini okur, 1
saniyelik pencerelere böler ve VERİ ŞEMASI'ndaki pencereler.parquet
sütunlarını hesaplar (toplam paket, mesaj türüne göre sayılar, farklı sysid
sayısı, seq atlama/tekrar, paketler arası süre ort/std, komut sayısı).
Pencere etiketi: pencerede en az bir saldırı paketi varsa o saldırı türü,
yoksa normal; attack_ratio da hesaplansın. Çıktı:
veri/oznitelikler/pencereler.parquet. src_ip/src_port/raw_hex KESİNLİKLE
öznitelik olmayacak.

3. bol.py: tüm uçuşları uçuş düzeyinde egitim/dogrulama/test olarak böler ve
veri/bolme.csv yazar (flight_id, split). Aynı uçuş tek kümede kalır.
Varsayılan ~%60/%20/%20. Saldırı türüne göre katmanlamaya çalış (her tür
mümkünse test'te de bulunsun); yeterli uçuş yoksa Türkçe uyar. Rastgelelik
için sabit tohum (seed) kullan, tekrar üretilebilir olsun.

4. dogrula.py: bir uçuş klasörünün eksiksiz olup olmadığını kontrol eder:
paketler.parquet var mı ve boş değil mi, iki yön de var mı, HEARTBEAT akıyor
mu, meta.json dolu mu, saldırılıysa ground_truth.json ve etiketli paketler
tutarlı mı (saldırı aralığında gerçekten saldırı etiketli paket var mı).
Türkçe bir rapor bassın: her uçuş için TAMAM/EKSİK ve nedeni. Tüm
veri/ucuslar'ı tarayan bir toplu mod da olsun.
```

> **Test:** `python betikler/etiketle.py --ucus F106_spoof_cmd` sonra parquet'te `label.value_counts()` ile saldırı etiketli paketleri gör. `python betikler/oznitelik.py --hepsi` → `pencereler.parquet` oluşmalı; sütunlar arasında src_ip/raw_hex OLMAMALI. `python betikler/bol.py` → `bolme.csv`. `python betikler/dogrula.py --hepsi` → hepsi TAMAM.

Adım 6 — Türkçe dokümantasyon

```
Repo için Türkçe belgeler yaz. Teknik bilgisi az biri bile takip edebilsin.

1. README.md (ana sayfa):
   - Proje nedir (2-3 cümle), bu reponun kapsamı (yalnızca veri üretimi)
   - Hızlı başlangıç: klonla, bash setup.sh, python betikler/kontrol.py
   - Bir normal ve bir saldırılı uçuş üretmenin tam komutları
(kopyalanabilir)
   - Klasör yapısı ve her betiğin bir cümlelik görevi (tablo)
   - Üç bilgisayar düzeni (pc_id ve numara aralıkları)
   - Etik/kapsam notu: saldırı betikleri yalnızca localhost; gerçek
sistemlerde kullanılmaz

2. belgeler/KURULUM.md: setup.sh ne yapıyor adım adım; Ubuntu 22.04 ve 24.04
farkları; WSL kullanılıyorsa notlar; s:k hatalar.

3. belgeler/VERI_URETIMI.md: bir uctan uca veri üretim oturumu — kaç
terminal, hangi sıra, SITL + proxy + ucus_yap + saldiri; normal ve saldırılı
uçuş adımları; önerilen uçuş listesi (en az 5 normal + 5 saldırılı, her
saldiri bir uçuş); hangi uçuşta hangi görev/rüzgâr.

4. belgeler/VERI_SEMASI.md: dosya ve sütun sözleşmesinin tamamı (bu projedeki
ortak sözleşme); veri sızıntısı uyarısı.

5. belgeler/SORUN_GIDERME.md: sim_vehicle.py bulunamadı, waf derleme hatası,
port dolu, harita açılmıyor (WSL), uçak AUTO'da kalkmıyor, parquet boş — her
biri için neden ve çözüm.

6. belgeler/VERI_PAYLASIMI.md: ham veri git'e konmaz; üç bilgisayardaki
uçuşlar nasıl tek yerde birleştirilir (ör. ortak bir Drive/sunucu klasörü),
dosya adları çakışmasın diye pc_id aralıkları.

Tüm belgeler Türkçe, kısa başlıklar, kopyalanabilir komut bloklarıyla.
```

> **Test:** `README.md`'yi baştan sona oku; bir başkası yalnızca bunu okuyarak kurup bir uçuş üretebilir mi? Eksik adım varsa Claude Code'a söyle.

Adım 7 — Test, ilk veri ve push

Önce uctan uca test

```
Şimdi her şeyi birlikte test edelim. Şu senaryoyu çalıştırmak için gereken
komutları bana sırayla ver ve her adımda ne görmem gerektiğini söyle:
1. SITL'yi başlat.
2. Bir normal uçuş üret (F101_normal), kaydet.
3. Bir saldırılı uçuş üret (F106_spoof_cmd), kaydet.
4. İkisini de etiketle, öznitelik çıkar, böl, doğrula.
Sonra dogrula.py çıktısını ve pencereler.parquet'in ilk birkaç satırını bana
göster.
```

Bu senaryo baştan sona sorunsuz çalışıyorsa repo hazır demektir. Çıkan hataları Claude Code ile düzelt.

Sonra en az 5 + 5 uçuş

Belgelerdeki önerilen listeye göre en az 5 normal (farklı görev/rüzgâr/irtifa; biri operatör komutlu) ve 5 saldırılı (her saldiri bir uçuş) üret. Bu, hocanın bu haftaki isteği.

Commit ve push

```
Tüm kodu ve belgeleri commit'le ve push'la. Ham veriyi (veri/ucuslar,
parquet, tlog, BIN) ASLA commit'leme; .gitignore bunu engelliyor olmalı, git
status ile teyit et. Anlamlı Türkçe commit mesajları kullan.
```

> **Test:** GitHub'da repoyu aç; kod ve belgeler görünüyor, ama `veri/ucuslar/` içindeki ağır dosyalar YOK. `git status` temiz olmalı.
>
> **Diğer iki arkadaş:** Repoyu klonlayıp yalnızca `bash setup.sh` ve `python betikler/kontrol.py` çalıştırsın. Hepsi TAMAM ise üç makinede de çalışıyor demektir. Her biri `config.yaml`'da kendi `pc_id`'sini ayarlasın (pc1/pc2/pc3).

Dikkat edilecekler ve sık sorunlar

Konu	Ne yapmalı
ArduPlane sürümü	Üç makinede de aynı olmalı: Plane-4.7.2. 4.7 ile bazı parametreler SRn_* yerine MAVn_* adıyla geçer; bu normal. Farklı sürüm kurarsanız sonuçlar tekrar üretilebilir olmaz.
--speedup kullanmayın	Simülasyonu hızlandırmak paketler arası süreleri bozar; zamanlama özniteliklerini buna dayanıyor. 1 kalsın.
Grafik arayüz	Ben (bu sohbetteki Claude) harita/konsol pencerelerini göremem; o yüzden SITL'yi kendi makinende çalıştırıp gözle. WSL'de harita açılmıyorsa `--map` olmadan çalış.
PyPI / pip	Sizin makinelerde pip normalde açık; setup.sh pymavlink ve MAVProxy'yi kurar. Bende (bulut) PyPI kapalı olduğu için SITL'yi burada test edemedim; bu yüzden kodu siz test ediyorsunuz.
Ham veri git'e girmesin	parquet/tlog/BIN dosyaları büyük; .gitignore dışlar. Her push'tan önce `git status` ile teyit et. Yanlışlıkla eklendiyse bana veya Claude Code'a söyle, geçmişten temizlensin.
Saldırı betikleri	Yalnızca localhost. Repo özel (private) olsa da betikler dış ağa çıkamayacak şekilde kilitli; bu kontrolü silmeyin.
Port dolu hatası	Önceki SITL/proxy kapanmamıştır. `pkill -f sim_vehicle; pkill -f kayit_proxy` ile temizle.
Üç kişi aynı anda	Her biri config.yaml'da kendi pc_id'sini ayarlasın (pc1/pc2/pc3). Böylece uçuş numaraları çakışmaz (F1xx/F2xx/F3xx).
Claude Code fazla şey yaparsa	"Sadece şu dosyayı düzenle, başkasına dokunma" de. Her adımdan sonra `git diff` ile ne değiştiğine bak.

Bu belge sizde kalıyor

Bu promptları ekiple paylaşabilirsin. Bir adımda takılırsan hatanın tam çıktısını bana getir; promptu düzeltir ya da sorunu çözeriz. Repo çalışır hale gelince, bir sonraki adım (tespit modeli, panel) için de aynı şekilde promptlar hazırlayabilirim.